SESSION 6

Kubernetes Observability

Metrics · Logs · Events · Prometheus · Grafana · Health Signals

Agenda

01 Introduction to Kubernetes Observability

02 Metrics in Kubernetes

03 Logs in Kubernetes

04 Events in Kubernetes

05 Prometheus — Architecture & Concepts

06 Grafana — Dashboards & Visualization

07 Kubernetes Health Signals

08 Alerting & Best Practices

What is Observability?

Observability is the ability to understand a system's internal state based on its external outputs.

Metrics

Numerical time-series data measuring system performance, resource usage, and business KPIs

Logs

Timestamped records of events that occurred inside your pods, containers, and cluster components

Traces

End-to-end tracking of requests across distributed microservices and service boundaries

In Kubernetes, observability covers the entire cluster: nodes, pods, containers, services, and the control plane.

Metrics in Kubernetes

Resource Metrics

CPU, memory, disk I/O, and network usage at node, pod, and container level via cAdvisor and kubelet

API Server Metrics

Request rates, latency, error counts, and etcd operations exposed at /metrics endpoint

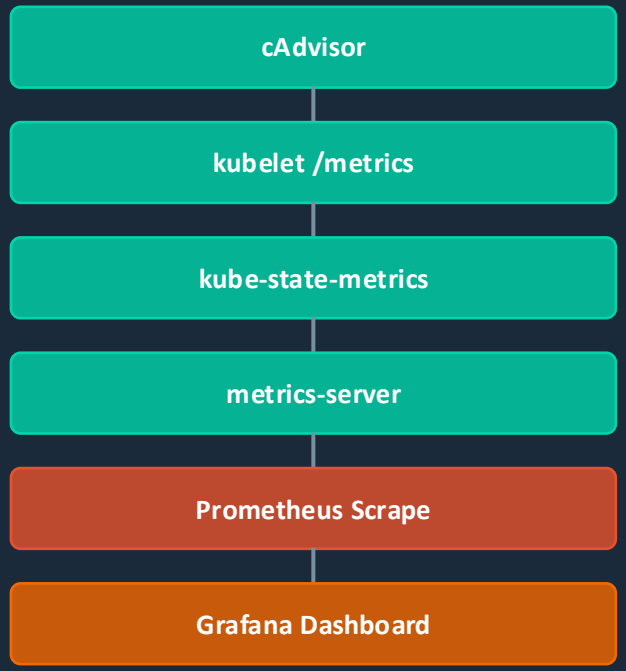
Custom Metrics

Application-defined metrics via Prometheus client libraries or external metrics adapters

State Metrics

Cluster object states (Deployments, Pods, Nodes) exposed by kube-state-metrics

Metrics Pipeline



Metrics Server & cAdvisor

Metrics Server

- Cluster-wide aggregator of resource usage data
- Collects CPU & memory from kubelet Summary API
- Required for kubectl top nodes/pods
- Powers HPA (Horizontal Pod Autoscaler)
- Short-term, in-memory storage only
- Not for long-term monitoring — use Prometheus

cAdvisor

- Container Advisor — built into kubelet
- Monitors resource usage per container
- Exposes metrics at /metrics/cadvisor
- CPU, memory, network, filesystem stats
- Integrates natively with Prometheus scraping
- Open source by Google — github.com/google/cadvisor

Logs in Kubernetes

Kubernetes Logging Architecture

Application Logs

stdout / stderr
from containers
kubectl logs <pod>

Node-Level Logging

logrotate manages
log files at node
/var/log/containers

Cluster-Level Logging

Centralized log
aggregation with
Fluentd / Loki

Popular Logging Tools

Fluentd

CNCF graduated project. Unified logging layer that collects, transforms & ships logs from all pods.

Fluent Bit

Lightweight log processor for Kubernetes. Low memory footprint, ideal for edge/embedded.

Loki

Grafana Labs log aggregation system. Indexes only labels — not full log content.

Elasticsearch

Part of EFK stack (Elasticsearch + Fluentd + Kibana). Full-text search across logs.

Events in Kubernetes

Kubernetes Events are objects that describe what happened to other Kubernetes objects. They are auto-generated by the API server.

Normal Events

- Pod scheduled to a node
- Container image pulled
- Pod started successfully
- Volume mounted successfully

Warning Events

- OOMKilled — out of memory
- BackOff — container crash looping
- Failed to pull image (ImagePullBackOff)
- Insufficient resources for scheduling

Useful kubectl Commands for Events

```
kubectl get events                                        // List all events in the current namespace
kubectl get events --sort-by=.metadata.creationTimestamp  // Sort events chronologically
kubectl describe pod <pod-name>                           // Show events for a specific pod
kubectl get events -A --field-selector type=Warning       // Show only Warning events cluster-wide
```

Prometheus — Overview

Prometheus

An open-source systems monitoring & alerting toolkit. Graduated CNCF project, originally built at SoundCloud in 2012.

Multi-dimensional data model:
Time-series identified by metric name + key/value label pairs

PromQL:
Flexible query language to aggregate data in real time

Pull-based model:
Scrapes targets over HTTP at defined intervals

Service Discovery:
Auto-discovers targets via Kubernetes, DNS, file-based configs

Alertmanager:
Handles deduplication, grouping, and routing of alerts

Key Stats

~50K
GitHub Stars

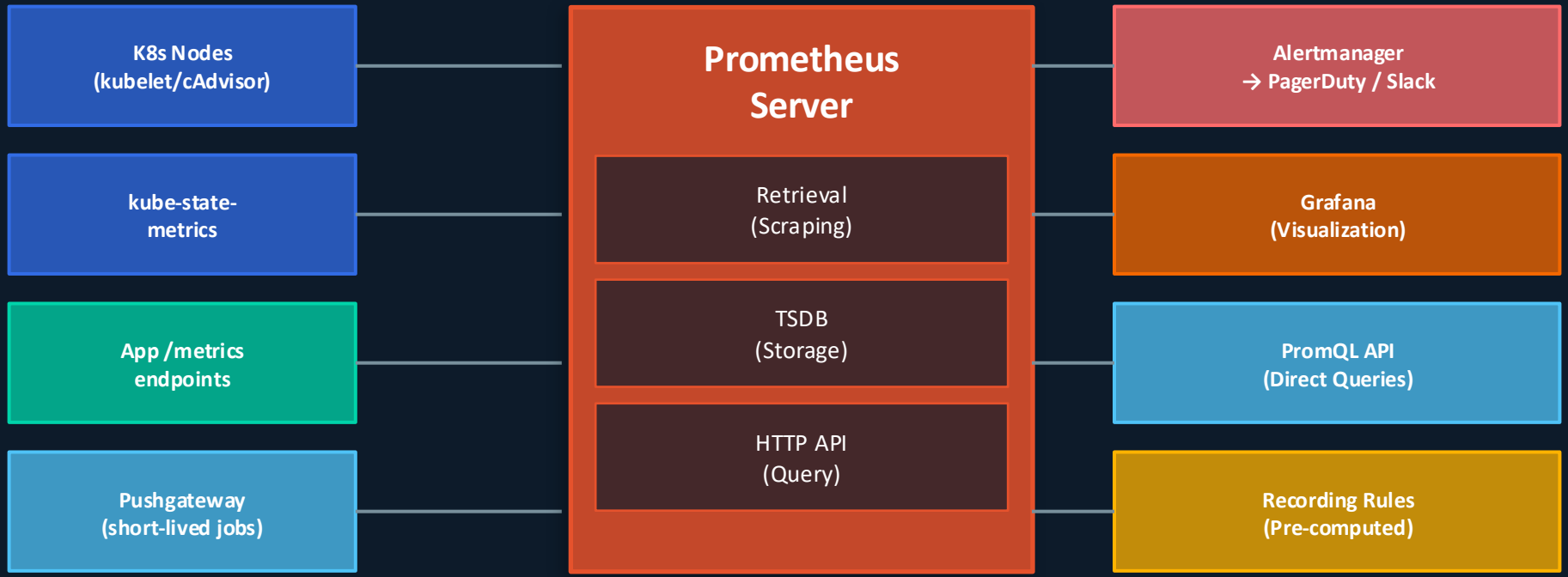
2012
Created at SoundCloud

2016
CNCF Joined

2018
CNCF Graduated

PromQL
Native Query Language

Prometheus Architecture



Service Discovery (Kubernetes SD / Static Config)

PromQL — Prometheus Query Language

PromQL is a functional query language that lets you select and aggregate time-series data in real time.

Instant Vector
A set of time series with a single sample for each — e.g. node_cpu_seconds_total

Range Vector
A set of time series with a range of data points — e.g. http_requests_total[5m]

Scalar
A single numerical floating point value

String
A simple string value (limited use currently)

Essential PromQL Examples

CPU Usage %	`sum(rate(container_cpu_usage_seconds_total[5m])) by (pod) * 100`
Memory Usage	`container_memory_working_set_bytes{namespace="default"} / 1024 / 1024`
Pod Restart Count	`increase(kube_pod_container_status_restarts_total[1h]) > 0`
HTTP Error Rate	`rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])`
Node Not Ready	`kube_node_status_condition{condition="Ready",status="true"} == 0`

Deploying Prometheus in Kubernetes

Method 1: Helm Chart

```
# Step 1
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
```

```
# Step 2

helm repo update
```

```
# Step 3
helm install prometheus prometheus-community/kube-prometheus-
stack
--namespace monitoring --create-namespace
```

```
# Step 4

kubectl get pods -n monitoring
```

```
# Step 5

kubectl port-forward svc/prometheus-operated 9090
```

Method 2: Prometheus Operator

ServiceMonitor

CRD that defines how to scrape Kubernetes services automatically

PodMonitor

CRD for scraping individual pods without a service

PrometheusRule

CRD that defines alerting and recording rules

Alertmanager

CRD to configure alert routing and receivers

ThanosRuler

For long-term metrics storage via Thanos integration

Grafana — Overview

Grafana is the open-source analytics and interactive visualization platform for observability data.

Dashboards

Build rich, interactive dashboards with 30+ visualization types including graphs, heatmaps, gauge panels

Data Sources

Supports 100+ data sources: Prometheus, Loki, InfluxDB, Elasticsearch, CloudWatch, and more

Alerting

Unified alerting system with multi-dimensional alerts, silence rules, and routing to multiple channels

Explore

Ad-hoc query interface for deep-dive investigation of metrics and logs without pre-built dashboards

Teams & RBAC

Organizations, teams, and role-based access control for enterprise multi-team environments

Annotations

Mark important events on graphs to correlate deployments, incidents, and metric changes

Grafana Dashboards & Panels

Panel Types

📈 Time series

Line/bar graph of metrics over time

🎯 Gauge

Single stat with threshold coloring

🔢 Stat

Big number display for KPIs

📋 Table

Tabular data with filtering/sorting

🌡️ Heatmap

Distribution over time buckets

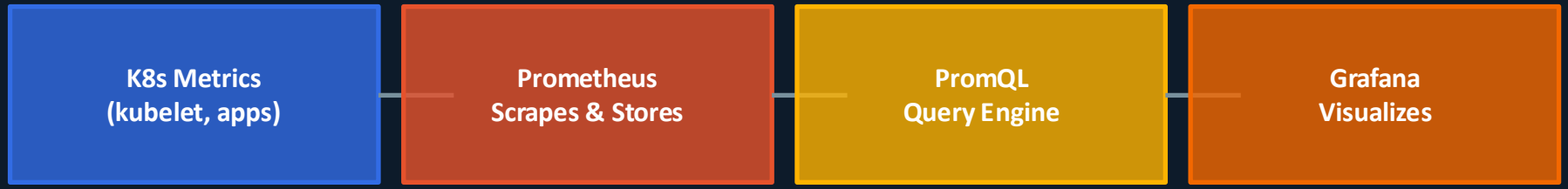
📝 Logs

Log lines from Loki data source

Popular Pre-built Kubernetes Dashboards

ID 315	Kubernetes cluster monitoring (via Prometheus)
ID 6417	Kubernetes Pods and Namespaces monitoring
ID 13332	Kubernetes kube-state-metrics
ID 12006	Kubernetes apiserver monitoring
ID 11074	Node Exporter Full dashboard

Grafana + Prometheus Integration



Connecting Grafana to Prometheus

1. Open Grafana → Configuration → Data Sources → Add data source
2. Select "Prometheus" and enter URL: http://prometheus-operated:9090
3. Set scrape interval (default: 15s), save & test connection
4. Import dashboard: Dashboards → Import → Enter dashboard ID (e.g. 315)
5. Select the Prometheus data source, click Import
6. Dashboard populates with live Kubernetes cluster metrics

Kubernetes Health Signals

Kubernetes exposes multiple health signals to determine the state of cluster components, nodes, and workloads.

Cluster Level

- API Server health (/healthz, /readyz, /livez)
- etcd cluster status & leader election
- Scheduler & controller manager health
- Node conditions: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable

Node Level

- Node Ready condition (kubelet heartbeat)
- CPU / Memory / Disk pressure conditions
- Node resource capacity vs allocatable
- Container runtime status (containerd / CRI)

Pod / Container Level

- Pod phase: Pending → Running → Succeeded/Failed
- Container state: Waiting, Running, Terminated
- Restart count & CrashLoopBackOff detection
- Liveness, Readiness, Startup probe results

Liveness, Readiness & Startup Probes

❤️ Liveness Probe

Purpose:

Is the container alive? If it fails, kubelet restarts the container.

When to use:

Use when your app can deadlock or hang without crashing.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
```

✅ Readiness Probe

Purpose:

Is the container ready to serve traffic? Fails = removed from Service endpoints.

When to use:

Use when app needs time to warm up (DB connections, caches).

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

🚀 Startup Probe

Purpose:

Has the container started yet? Disables liveness/readiness until it succeeds.

When to use:

Use for slow-starting containers to avoid premature restarts.

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

Node Conditions & Pod Phases

Node Conditions

Condition	Healthy Value	Description
`Ready`	True = healthy	Node is healthy and ready to accept pods
`MemoryPressure`	True = healthy	Node is running low on available memory
`DiskPressure`	True = healthy	Node disk capacity is low or inode count is low
`PIDPressure`	True = healthy	Too many processes running on the node
`NetworkUnavailable`	True = healthy	Node network not correctly configured

Pod Phases

Pending

Accepted but not yet scheduled/started

Running

At least one container is running

Succeeded

All containers terminated successfully

Failed

At least one container failed

Unknown

Cannot determine pod state

kube-prometheus-stack

The kube-prometheus-stack Helm chart provides a complete end-to-end Kubernetes monitoring solution out-of-the-box.

Prometheus Operator

Manages Prometheus, Alertmanager, and related components via CRDs

Prometheus

Metrics collection and storage with pre-configured Kubernetes scrape jobs

Alertmanager

Alert routing, grouping, and notification to Slack, PagerDuty, email

Grafana

Pre-built dashboards for cluster, node, pod, and namespace monitoring

Node Exporter

Exposes hardware and OS-level metrics from each node (CPU, disk, network)

kube-state-metrics

Generates metrics about Kubernetes object states (Deployments, Pods, etc.)

Observability Stack Comparison

Tool	Type	Best For	Storage
Prometheus	Metrics	Pull-based metrics scraping & alerting	TSDB (local)
Grafana	Visualization	Dashboards for metrics, logs, traces	None (uses sources)
Loki	Logs	Log aggregation — label-indexed only	Object storage
Jaeger	Traces	Distributed request tracing across services	Cassandra / ES
Thanos	Long-term Metrics	Global Prometheus view + long-term storage	Object storage
OpenTelemetry	All signals	Vendor-neutral instrumentation & collection	Pluggable

Observability Best Practices

01

USE Method

For every resource: Utilization, Saturation, Errors. Covers CPU, memory, disk, network.

02

RED Method

For services: Rate, Errors, Duration. Covers request-oriented metrics for microservices.

03

4 Golden Signals

Latency, Traffic, Errors, Saturation — from Google SRE. The basis for SLA/SLO monitoring.

04

Label Consistently

Use consistent labels: cluster, namespace, pod, container, env. Avoid high-cardinality labels.

05

Set Retention Policies

Local Prometheus: 15d default. Use Thanos/Cortex for 90d+ long-term metrics storage.

06

Alert on Symptoms

Alert on user-visible problems (latency, errors), not causes. Avoid alert fatigue.

07

Structure Logs as JSON

Use structured logging (JSON) for easy filtering and indexing in Loki/Elasticsearch.

08

Correlate Signals

Use trace IDs in logs, link Grafana panels with Loki for unified root-cause analysis.

Summary & Key References

Session Summary

- Kubernetes observability rests on Metrics, Logs, and Events as three pillars
- Prometheus scrapes metrics via pull model; PromQL enables flexible querying
- Grafana connects to Prometheus for rich dashboards and alerting visualization
- Health signals include probes (Liveness/Readiness/Startup) and Node conditions
- kube-prometheus-stack provides a complete production-ready monitoring bundle
- Alert on symptoms using RED/USE methods and the 4 Golden Signals framework

Official Documentation

Kubernetes Docs	kubernetes.io/docs
Prometheus	prometheus.io/docs
Grafana	grafana.com/docs
kube-state-metrics	github.com/kubernetes/kube-state-metrics
cAdvisor	github.com/google/cadvisor
Loki	grafana.com/oss/loki
OpenTelemetry	opentelemetry.io
Fluentd	fluentd.org/architecture
Node Exporter	github.com/prometheus/node_exporter
SRE Book	sre.google/sre-book